

**Universitatea
Transilvania
din Brașov**

**FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR**

PROIECT

**Conducător științific:
Attila Pal**

**Studenti:
Nechifor Cristina
Runceanu Vlad
Munteanu Daniel Ioachim
Stoian Dana Alexandra**

BRAȘOV, 2026

Departamentul

Programul de studii:

Dezvoltarea unei aplicații web interactive de tip Paint

Conducător științific:

Attila Pal

Lista de figuri, tabele și coduri sursă.....	4
Lista de acronime	5
1 Introducere	6
1.1. Contextul proiectului.....	6
1.2. Motivația alegerii temei	6
1.3. Obiectivele proiectului	6
2 Fundamente teoretice	7
1.4. Arhitectura aplicației	7
1.5. Cerințe funcționale (Use Cases).....	8
1.6. Diagrama de flux a aplicației	9
3 Proiectare și implementare.....	11
3.1. Arhitectura generală a sistemului.....	11
3.1.1 Funcționarea generală a sistemului	11
3.1.2 Diagrama arhitecturii sistemului	11
3.1.3 Fluxul procesului general	12
3.1.4 Componentele software.....	13
3.1.5 Componentele hardware.....	15
3.2 Implementare	16
3.2.1 Structura fișierelor și organizarea proiectului	16
3.2.2 Configurarea suprafeței de lucru (Canvas API)	16
3.2.3 Definirea elementelor interfeței	17
3.2.4 Estetica interfeței (CSS).....	18
3.2.5 Logica de funcționare și interactivitatea	20
3.2.6 Funcții avansate.....	21
4 Testare.....	23
4.1 Verificarea funcționalităților.....	23
4.2 Testarea Automată	23
4.3 Rezultate.....	25
4.3.1 Testarea grafică	25
4.3.2 Testarea codului	26
4.4 Mentenanță și dezvoltare	27
5 Concluzii.....	29
6 Bibliografie	31

LISTA DE FIGURI, TABELE ȘI CODURI SURSĂ

FIGURI

Figură 1. Diagrama de flux.....	10
Figură 2. Arhitectura Aplicației.....	12
Figură 3. Interfața spațiului de lucru.....	14
Figură 4. Setări inițiale.....	17
Figură 5. Exemplu configurare.....	18
Figură 6. Efecte pentru butoane.....	19
Figură 7. Logica de funcționare.....	21
Figură 8. Mutarea obiectelor pe canvas.....	22
Figură 9. Verificare stări.....	24
Figură 10. Gestionarea memoriei.....	24
Figură 11. Funcția undo după 20 de accesări.....	25
Figură 12. Interfața și testarea grafică.....	26
Figură 13. Raportul final al testelor.....	27
Figură 14. Logo-ul aplicației.....	30

TABELE

Tabelul 1. Use Cases.....	8
---------------------------	---

- API- Application Programming Interface
- CSS- Cascading Style Sheets
- DOM – Document Object Model
- HTML- HyperText Markup Language
- IDE – Integrated Development Environment
- JS- JavaScript
- PNG – Portable Network Graphics
- RGB- Red Green Blue
- UI- User Interface
- UX- User Experience

1 INTRODUCERE

1.1. Contextul proiectului

În prezent, majoritatea aplicațiilor se mută din zona de desktop direct în browser, iar acest lucru este posibil datorită evoluției standardelor web. Ca studenți la inginerie, am observat că avem nevoie de instrumente de schițare rapide care să nu depindă de un sistem de operare anume. Proiectul de față a fost dezvoltat pentru a demonstra cum putem folosi tehnologiile de frontend pentru a crea o unealtă de desen interactivă, accesibilă de pe orice dispozitiv cu un browser modern.

1.2. Motivația alegerii temei

Motivația principală a fost curiozitatea de a vedea cum putem gestiona fluxuri de date grafice în timp real folosind JavaScript și elementul Canvas din HTML5. Deși pare un simplu program de desenat, implementarea funcțiilor de bază ridică probleme interesante de programare și logică, specifice profilului nostru de inginerie.

1.3. Obiectivele proiectului

Prin acest proiect, ne-am propus să realizăm o aplicație care să permită următoarele:

- Interacțiune fluidă: Desenarea fără lag prin captarea evenimentelor de mouse pe suprafața activă a paginii.
- Set de unelte esențiale: Implementarea funcțiilor de tip creion și radieră.
- Controlul parametrilor: Posibilitatea de a modifica dinamic culorile și dimensiunea creionului.
- Exportul rezultatului: Salvarea desenului final sub format imagine, pentru a putea fi utilizat în alte documente.

2 FUNDAMENTE TEORETICE

1.4. Arhitectura aplicației

Aplicația este construită pe o arhitectură de tip client-side, ceea ce înseamnă că toată logica de desenare și procesare a imaginii are loc direct în browser-ul utilizatorului. Arhitectura aleasă oferă o viteză de răspuns foarte mare, nefiind nevoie de comunicare permanentă cu un server pentru a randa o linie pe ecran. Structura proiectului este una modulară, împărțită pe trei niveluri:

1. Structura (HTML5):

Acest nivel reprezintă scheletul aplicației și definește ierarhia elementelor vizibile. Elementul central este tag-ul `<canvas>`, care acționează ca o suprafață grafică programabilă. Pe lângă acesta, am creat secțiuni separate pentru butoane și setări.

Am organizat totul astfel încât interfața să fie intuitivă: butoanele pentru unelte sunt grupate într-o parte, iar zona de desen ocupă restul spațiului. Această separare ne ajută să avem un cod ordonat și ușor de modificat dacă vrem să mai adăugăm butoane noi pe viitor.

2. Design-ul (CSS3)

Acest nivel se ocupă de aspect. Am ales un fundal închis pentru ca aplicația să arate modern și să fie odihnitoare pentru ochi. Folosind CSS, am stabilit dimensiunile butoanelor, culorile lor și modul în care acestea se așază în pagină.

Ne-am asigurat că butoanele reacționează când treci cu mouse-ul peste ele (se schimbă culoarea sau apare o umbră), oferind utilizatorului un semnal clar că poate da click pe ele. De asemenea, am setat designul astfel încât elementele să nu se suprapună și să rămână aliniate frumos, indiferent de mărimea ferestrei browserului.

3. Logica (JavaScript)

JavaScript-ul monitorizează în permanență ce face utilizatorul cu mouse-ul. Când apăsăm pe butonul mouse-ului, programul știe că trebuie să înceapă desenarea, iar când mișcăm mouse-ul, calculează poziția exactă și trage o linie.

Tot aici am scris codul care ține minte pașii făcuți, pentru ca butonul de Undo să funcționeze corect. Practic, JavaScript face legătura între butoanele de pe ecran și ceea ce apare desenat pe canvas, asigurând o mișcare fluidă și fără întârzieri.

1.5. Cerințe funcționale (Use Cases)

Pentru a stabili ce trebuie să facă aplicația PaintStudio, am pornit de la funcțiile de bază ale unui editor grafic clasic. Am vrut ca interfața să fie simplă și să permită utilizatorului să găsească repede uneltele de care are nevoie. Am împărțit funcționalitățile în câteva categorii clare: alegerea culorilor, reglarea dimensiunii urmei lăsate pe canvas, unelte de corecție și gestionarea fișierului final.

Toate aceste acțiuni pe care utilizatorul le poate face în aplicație sunt centralizate în tabelul de mai jos, servind ca bază pentru scrierea scripturilor JavaScript.

Tabel 1. Use Cases

Nr. Crt.	Funcționalitate	Descriere
1.	Desenare liberă	Este funcția principală a programului și permite trasarea de linii folosind creionul prin apăsarea mouse-ului în mod continuu.
2.	Selecție culori	Utilizatorul are la dispoziție un set din care poate alege o culoare sau o nuanță pentru schimbarea liniei de desen.
3.	Reglare grosime	Poate fi setată grosimea vârfului creionului, în funcție de nevoile utilizatorului. Prin slider-ul care permite modificarea rapidă și eficientă a grosimii, se asigură ușurința realizării detaliilor.
4.	Control mărime canvas (zoom)	Butoanele de +/- permit modificarea mărimii spațiului de lucru, ceea ce ajută utilizatorul să aibă precizie mult mai bună.
5.	Funcție undo	Un sistem care face posibilă revenirea la pasul anterior, fără a fi nevoiți să ștergem părțiri întregi, cu riscul de a elimina și altceva.
6.	Ștergere (radieră)	Această funcție oferă utilizatorului posibilitatea de a șterge anumite porțiuni din desen. Asemenea creionului, grosimea ei poate fi modificată, permițând ștergerea unor zone mai mari/mici.

7.	Resetare	Utilizatorul poate începe toată lucrarea de la zero, prin revenirea la canvasul inițial.
8.	Export desen	La finalizare, desenul poate fi salvat sub forma unui fișier .png, o imagine care poate fi păstrată sau folosită extern.

1.6. Diagrama de flux a aplicației

Pentru a înțelege cum funcționează aplicația de desen, am realizat o diagramă de flux care urmărește pașii logici de la pornire până la oprire. Logica se bazează pe monitorizarea mouse-ului în zona de desenare:

1. Pornirea sistemului (START)

În momentul în care aplicația este încărcată în browser, programul intră într-o stare de așteptare. JavaScript activează senzorii *event listeners* care urmăresc orice mișcare sau apăsare pe suprafața canvas-ului.

2. Așteptare eveniment mousedown

Acesta este punctul critic unde aplicația verifică dacă utilizatorul a apăsă butonul mouse-ului. Dacă nu există nicio apăsare, programul rămâne în buclă, așteptând interacțiunea.

3. Procesarea desenului (Mouse apăsă)

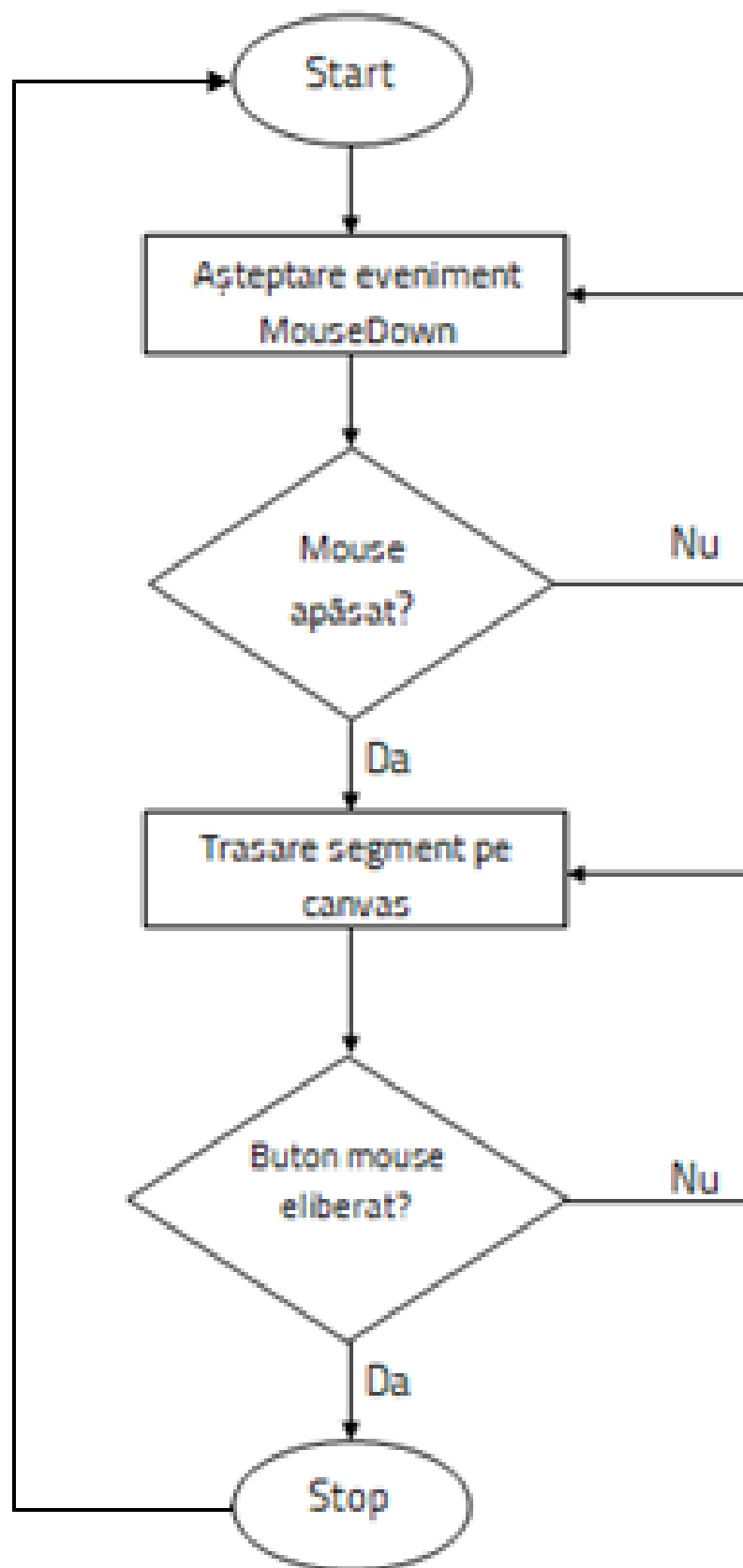
Odată ce butonul este detectat ca fiind apăsă, aplicația trece la etapa de trasare. Cât timp mouse-ul se mișcă pe canvas, programul calculează rapid noile coordonate X și Y și le unește pentru a forma o linie continuă pe ecran.

4. Verificarea stării (Buton eliberat)

Programul verifică în permanență dacă degetul a fost ridicat de pe butonul mouse-ului. Atâta timp cât butonul rămâne apăsă, desenarea continuă fără întrerupere .

5. Finalizarea acțiunii (STOP)

Când butonul mouse-ului este eliberat, procesul de trasare se oprește imediat. Aplicația revine în starea inițială de așteptare, fiind pregătită pentru o nouă sesiune de desenare sau pentru folosirea butoanelor de control.



Figură 1. Diagrama de flux

3 PROIECTARE ȘI IMPLEMENTARE

3.1. ARHITECTURA GENERALĂ A SISTEMULUI

Urmărind procedeul de proiectare al aplicației rezultă ca produs final un instrument care, utilizând browser-ul, face posibilă economisirea resurselor sistemului, prin evitarea instalării locale a software-ului fapt ce atestă utilizarea tehnologiilor Web HTML, CSS și JavaScript.

Obiectivul principal a fost ca aplicația să fie rapidă și să reacționeze imediat la mișcările mouse-ului, oferind în același timp o interfață simplă.

3.1.1 Funcționarea generală a sistemului

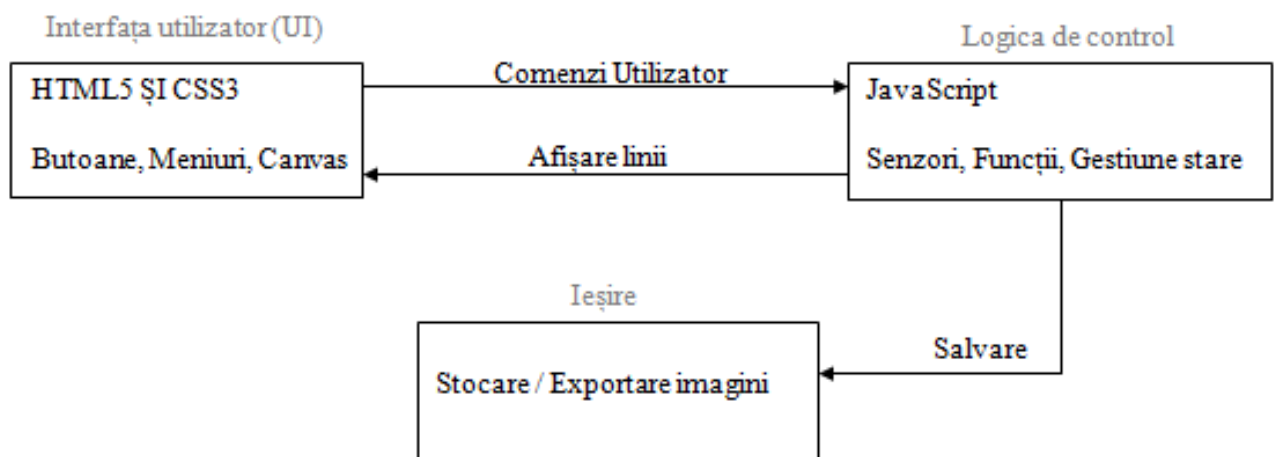
Aplicația a fost concepută ca o soluție web interactivă, sistemul funcționând pe un model bazat pe straturi:

1. Interfața de control- reprezintă zona superioară, unde sunt vizibile toate uneltele. Fiecare buton sau slider este legat de o funcție specifică din codul JavaScript.
2. Logica de control- este motorul aplicației, responsabil pentru procesarea acțiunilor. Coordonează programul, interpretează comenzile și le transformă în instrucțiuni, în funcție de acțiunile utilizatorului.
3. Suprafața de desen- este elementul central unde utilizatorul vede ceea ce desenează, în timp real. Transformă mișcările mouse-ului în coordonate și apoi în linii colorate.

3.1.2 Diagrama arhitecturii sistemului

Pentru a vizualiza mai ușor traseul informației, am realizat o diagramă, care arată drumul parcurs de la mișcarea mouse-ului până la afișarea rezultatului pe ecran.

Utilizatorul interacționează cu interfața, care trimite comanda către logica de control, iar aceasta desenează pe canvas. Rezultatul final este cel pe care utilizatorul îl vede instantaneu în browser.



Figură 2. Arhitectura Aplicației

Schema de mai sus ne ajută să înțelegem cum decurge comunicarea între cele trei module principale.

Totul începe de la interfața grafică (UI), unde comenzile utilizatorului sunt recepționate sub formă de semnale brute, cum ar fi coordonatele mouse-ului sau click-urile pe butoanele de culori. Aceste date de la interfață sunt trimise imediat către logica de control, unde JavaScript preia rolul de „translator” și decide ce trebuie să se întâmple cu ele.

De exemplu, dacă noi am selectat radiera, logica de control procesează această alegere și instruește sistemul să schimbe culoarea urmei în alb, indiferent de nuanța folosită anterior.

După ce aceste instrucțiuni sunt procesate, rezultatul este trimis către suprafața de desen, unde liniile și formele apar instantaneu pe ecran.

În final, dacă decidem că am terminat lucrul, procesul se încheie prin modulul de stocare și ieșire, care preia toți pixelii de pe canvas și îi assemblează într-un fișier imagine ce poate fi descărcat direct în calculator.

3.1.3 Fluxul procesului general

Fluxul de funcționare al aplicației a fost structurat să asigure o interacțiune fluidă între utilizator și mediul de desenare. Procesul general urmează pașii reprezentați anterior în capitolul de fundamente teoretice, fiind susținut de un ciclu continuu de captare a datelor.

Procesul începe prin afișarea Canvasului, imediat ce pagina este încărcată în browser. Din acest moment, aplicația intră într-o stare de „ascultare”, așteptând ca utilizatorul să interacționeze cu interfața.

Fluxul poate fi împărțit în trei mari etape:

1. Etapa de configurare- utilizatorul selectează instrumentele de lucru , fiecare alegere modificând starea variabilelor interne ale programului.
2. Etapa de procesare grafică- în momentul în care mouse-ul este detectat ca fiind apăsat, se declanșează fluxul de desenare. Aplicația calculează în timp real distanța dintre poziția anterioară și cea curentă a cursorului, generând segmente de linie care sunt afișate instantaneu.
3. Etapa de finalizare și stocare- la eliberarea butonului de mouse, procesul de desenare se oprește, iar starea curentă a canvas-ului este salvată într-un istoric sau poate fi exportată ca fișier imagine prin procesul de salvare.

Acest flux circular permite utilizatorului să revină oricând la etapa de configurare pentru a schimba parametrii desenului, fără a întrerupe funcționarea de bază a sistemului.

3.1.4 Componentele software

Pentru realizarea proiectului, au fost selectate instrumente software care să asigure un mediu de dezvoltare eficient . Alegerea acestora s-a bazat pe standardele actuale în dezvoltarea aplicațiilor web.

1. Sistemul de operare utilizat ca platformă principală este Windows 10/11.

Acesta ne-a oferit mediul stabil de care am avut nevoie pentru a rula în același timp editorul de cod și mai multe ferestre de browser pentru teste. Totuși, un avantaj major al tehnologiilor web alese este independența de platformă, aplicația putând fi dezvoltată sau rulată la fel de bine pe macOS sau Linux.

2. Pentru mediul de dezvoltare am optat pentru Visual Studio Code.

Am ales acest editor deoarece ne permite să organizăm fișierul HTML, cel de CSS și cel de JavaScript într-un mod vizual foarte clar.

De asemenea, folosim extensia *Live Server* pentru a vizualiza în timp real modificările aduse codului. Aceasta actualizează automat pagina de fiecare dată când salvăm, fără a fi nevoie de reîncărcarea manuală a browserului la fiecare pas.

3. Principalul instrument folosit pentru testare a fost Google Chrome.

Îl folosim datorită utilitarului *Chrome DevTools*. Acesta ne permite să inspectăm elementele din DOM, să monitorizăm performanța elementului Canvas și să depanăm erorile din scripturile JavaScript, folosind consola.

4. Limbaje și biblioteci:

- HTML5-este folosit pentru definirea structurii de bază a aplicației, oferind foaia de lucru prin elementul canvas și containerele necesare pentru organizarea butoanelor.
- CSS3- limbajul ne permite stilizarea interfeței aplicației și este folosit pentru stilizarea paginii, implementarea temei întunecate și aranjarea responsivă a butoanelor.
- JavaScript- fiind nucleul proiectului, este folosit pentru toată logica de calcul a coordonatelor, gestionarea memoriei pentru funcția undo.

```
<div class="studio">

  <div class="toolbar">
    <span class="toolbar-label">Culori</span>
    <div id="colors">
      <button id="black" onclick="changeColor('#1a1a2e')" title="Negru"></button>
      <button id="white" onclick="changeColor('#f0f0f5')" title="Alb"></button>
      <button id="red" onclick="changeColor('#ff4757')" title="Roșu"></button>
      <button id="pink" onclick="changeColor('#ff6b9d')" title="Roz"></button>
      <button id="orange" onclick="changeColor('#ff9f43')"
title="Portocaliu"></button>
      <button id="yellow" onclick="changeColor('#ffd93d')" title="Galben"></button>
      <button id="green" onclick="changeColor('#2ed573')" title="Verde"></button>
      <button id="teal" onclick="changeColor('#1e90ff')" title="Cyan"></button>
      <button id="blue" onclick="changeColor('#5352ed')"
title="Albastru"></button>
      <button id="purple" onclick="changeColor('#a55eea')" title="Violet"></button>

      <div class="custom-picker-wrapper" id="customPickerWrapper">
```

```

    <button id="customColorBtn" onclick="toggleCustomPicker()" title="Culoare
Personalizată"></button>
    <div class="custom-picker-panel" id="customPickerPanel">

        <div class="native-picker-wrapper" title="Deschide spectrul de culori">
            <input type="color" id="nativeColorInput" value="#1a1a2e"
oninput="handleNativeColor(this.value)">
            <span class="native-picker-hint">Apasă pentru spectru</span>
        </div>

        <div class="picker-hex">
            <span class="hex-symbol">#</span>
            <input type="text" id="hexInput" maxlength="6" placeholder="Hex Code">
            <button class="apply-btn" onclick="applyHexInput()">OK</button>
        </div>
    </div>
</div>
</div>

```

Figură 3. Interfața spațiului de lucru

3.1.5 Componentele hardware

Deoarece PaintStudio este o aplicație web, resursele hardware necesare sunt minime, execuția bazându-se pe resursele sistemului de calcul al utilizatorului final.

Acest lucru reprezintă un avantaj, deoarece programul nu depinde de puterea unui server extern, ci folosește componentele locale ale calculatorului pentru a randa desenele.

Pentru o experiență de utilizare optimă, se recomandă un sistem dotat cu minim 4GB RAM deoarece memoria este intens folosită pentru a stoca stările succesive ale desenului în cadrul funcției de undo. Cu cât memoria este mai rapidă, cu atât mai mulți pași pot fi salvați și recuperați instantaneu fără a încetini browserul.

Este necesar un procesor capabil să susțină un browser modern, precum Chrome, Edge sau Firefox, asigurând fluiditatea mișcărilor pe canvas. Procesorul are rolul de a calcula coordonatele X și Y de zeci de ori pe secundă, transformând mișcarea fizică a mouse-ului în linii continue pe ecran fără întreruperi.

Fiind o aplicație grafică, am luat în calcul și dispozitivele de intrare sau afișare, deoarece calitatea monitorului influențează direct modul în care sunt percepute culorile alese din paletă. De asemenea, fluiditatea desenului depinde de precizia mouse-ului, sistemul nostru fiind optimizat să interpreteze semnalele de intrare fără întârziere.

3.2 IMPLEMENTARE

3.2.1 Structura fișierelor și organizarea proiectului

Aplicația este compusă din trei fișiere principale, fiecare având un rol specific în ecosistemul proiectului:

1. INDEX.html

Definește structura scheletică a paginii. Aici am creat containerele principale pentru bara de unelte și elementul de tip canvas care reprezintă spațiul propriu-zis de desenare.

Tot în acest fișier am realizat legăturile către foaia de stil și către scriptul de logică, asigurând comunicarea între componente.

2. CSS4.css

Gestionează aspectul vizual și poziționarea elementelor. Prin utilizarea CSS, am reușit să implementăm un design responsiv, care se adaptează în funcție de mărimea ecranului, asigurând o interfață intuitivă pentru utilizator.

3. JS.js

Reprezintă creierul aplicației. Aici este implementată logica pentru manipularea pixelilor pe canvas, gestionarea evenimentelor de mouse și sistemul de undo. Fișierul JavaScript preia acțiunile utilizatorului din HTML și le transformă în instrucțiuni de desenare, fiind elementul care face aplicația să devină interactivă.

Separarea proiectului în aceste trei fișiere ne-a permis să lucrăm mult mai eficient în cadrul echipei. De exemplu, am putut ajusta culorile butoanelor în fișierul CSS fără a fi nevoie să modificăm logica de calcul a coordonatelor din JavaScript.

Această abordare modulară este esențială pentru mentenanța proiectului, facilitând adăugarea de noi funcționalități pe viitor fără a complica structura existentă.

3.2.2 Configurarea suprafeței de lucru (Canvas API)

Ca să putem desena, codul JavaScript trebuie mai întâi să găsească zona din pagină unde se află Canvas-ul. Acest proces este esențial deoarece, fără o legătură corectă între script și elementul vizual din HTML, nicio comandă de desenare nu ar putea fi executată.

Identificăm Canvas-ul prin instrucțiunea *getElementById*, care îi spune programului să lucreze exact cu elementul pe care l-am numit „canvas” în structura HTML. După ce am stabilit această legătură, trebuie să alegem modul de desen.

Folosim comanda *getContext('2d')*, ceea ce înseamnă că lucrăm pe o suprafață plană, bidimensională, unde putem controla fiecare pixel prin coordonate matematice.

Tot în această etapă de setări inițiale, am configurat ca liniile să aibă capetele rotunjite prin proprietatea *lineCap*, pentru ca desenele să aibă un aspect fluid și să nu pară pătrătoase atunci când schimbăm direcția mișcării.

Un aspect important pe care l-am adăugat în această configurare este opțiunea *willReadFrequently: true*. Am ales să folosim această setare pentru a optimiza performanța aplicației, deoarece sistemul nostru de undo și funcția de salvare a imaginii necesită citirea repetată a pixelilor de pe canvas.

Prin această instrucțiune, anunțăm browserul să pregătească memoria într-un mod care să permită accesul rapid la datele grafice, asigurând astfel că aplicația rămâne rapidă chiar și după ce am realizat un desen complex cu foarte multe detalii.

```
canvas = document.getElementById('canvas');
canvasWrapper = document.getElementById('canvasWrapper');
// Cerem browserului sa ne lase sa desenam in format 2D
ctx = canvas.getContext('2d', { willReadFrequently: true });
```

Figură 4 Setări inițiale

3.2.3 Definirea elementelor interfeței

După configurarea tehnică a mediului de lucru, a fost creată structura vizuală în fișierul INDEX.html. Acesta conține toate meniurile, butoanele și zonele cu care utilizatorul interacționează direct.

Elementele cheie aici sunt antetul, panoul de control, bara de stare și zona de desen.

- Antetul și Panoul de control

Antetul aplicației are rolul de a identifica proiectul, conținând logo-ul și titlul „Paint Studio”. Imediat sub acesta, am implementat panoul de control, care oferă utilizatorului acces rapid la toate instrumentele disponibile.

Aici am grupat selectorul de culori, butoanele pentru creion și radieră, dar și funcțiile de sistem precum undo sau salvare. Fiecare buton a fost marcat clar, astfel încât rolul său să fie evident de la prima utilizare.

- Bara de stare și Zona de desen

Zona de desen reprezintă elementul central, funcționând ca o foaie virtuală pe care utilizatorul o folosește pentru a crea linii și desene complexe.

În completarea ei, am adăugat o bară de stare care afișează informații utile în timp real, cum ar fi poziția exactă a mouse-ului. Această funcționalitate ajută utilizatorul să aibă o precizie mai mare atunci când vrea să înceapă un desen dintr-un punct specific sau când vrea să măsoare vizual spațiul de lucru.

Pentru a oferi un control cât mai fin asupra instrumentelor, am integrat elemente de interfață interactive, cum este exemplul de configurare pentru grosimea liniei prezentat în codul de mai jos. Am utilizat un element de tip input range (slider), care îi permite utilizatorului să modifice mărimea urmei lăsate pe canvas între o valoare minimă de 1 și una maximă de 40.

Această componentă este legată direct de o funcție numită `changeSize`, care preia valoarea selectată în timp real și actualizează atât grosimea creionului, cât și indicatorul vizual de pe ecran. Prin această metodă, utilizatorul vede imediat cât de groasă va fi linia înainte de a începe să deseneze efectiv, evitând astfel erorile de trasare.

```
<div class="toolbar">
  <span class="toolbar-label">Grosime</span>
  <div class="size-control">
    <input type="range" class="size-slider" id="sizeSlider" min="1" max="40"
value="5" oninput="changeSize(this.value)">
    <div class="size-preview">
      <div class="size-dot" id="sizeDot" style="width:5px;height:5px;"></div>
    </div>
    <span id="sizeValueDisplay" class="size-value">5px</span>
  </div>
</div>
```

Figură 5. Exemplu configurare

3.2.4 Estetica interfeței (CSS)

Principalul obiectiv a fost crearea unei interfețe de tip Dark Mode, care reduce oboseala ochilor și pune în evidență culorile desenului. Întregul cod pentru aspectul vizual se găsește în fișierul `CSS4.css`, unde am definit reguli care asigură un echilibru între estetică și funcționalitate.

Elementele principale de design pe care ne-am concentrat sunt variabilele de sistem, flexibilitatea elementelor și feedback-ul vizual instantaneu.

Am utilizat variabilele CSS pentru a putea controla paleta de culori a întregului proiect dintr-un singur loc. Această metodă ne-a permis să obținem un aspect uniform și ne oferă posibilitatea de a schimba întreaga temă a aplicației prin modificarea câtorva linii de cod, fără a căuta prin sute de reguli de stilizare.

În ceea ce privește flexibilitatea, am configurat panourile și butoanele astfel încât să fie responsive. Indiferent dacă utilizatorul deschide aplicația pe un monitor mare sau pe un laptop cu ecran mic, interfața se adaptează automat, păstrând butoanele la dimensiuni utilizabile și zona de desen centrală.

Un aspect esențial pentru experiența utilizatorului este feedback-ul vizual, pe care l-am implementat prin animații subtile și schimbări de stare. În codul prezentat în Figura 6, se poate observa cum am definit comportamentul butoanelor de selecție a culorilor.

Am folosit proprietatea `transition` pentru a crea o trecere lină (de 0.15 secunde) atunci când mouse-ul trece peste un buton (starea `:hover`). Acest efect de mărire (`scale(1.2)`) și adăugarea unei umbre discrete (`box-shadow`) îi confirmă utilizatorului că elementul este interactiv.

De asemenea, am definit o clasă specială numită `.active`, care se aplică butonului selectat în momentul respectiv. Aceasta adaugă o bordură albă și o umbră mai pronunțată, permițând utilizatorului să știe în permanență ce culoare sau unealtă are activă fără a fi nevoie să verifice alte meniuri.

Aceste detalii de design nu doar că fac aplicația să arate modern, dar îmbunătățesc semnificativ accesibilitatea, transformând utilizarea programului într-o interacțiune intuitivă și plăcută.

```
/* Meniul de culori predefinite */
#colors {
  display: flex;
  align-items: center;
  gap: 8px;
  flex-wrap: wrap;
}

#colors > button {
  width: 30px;
  height: 30px;
  border-radius: 50%;
  border: 2px solid transparent;
  cursor: pointer;
  transition: transform 0.15s ease, box-shadow 0.15s ease; /* Animație lină la interacțiune */
  outline: none;
}
```

```
#colors > button:hover {
  transform: scale(1.2);
}

#colors > button.active {
  border-color: white;
  box-shadow: 0 0 0 3px rgba(255, 255, 255, 0.25); /* Creează un inel vizual extern */
  transform: scale(1.15);
}
```

Figură 6. Efecte pentru butoane

3.2.5 Logica de funcționare și interactivitatea

Creierul aplicației, care se ocupă de logica de funcționare și interactivitate în timp real, este JavaScript-ul. Acesta gestionează modul în care utilizatorul desenează procesează comenzile primite de la interfață și coordonează sistemul de memorie.

Principalele mecanisme care sunt implementate, pentru o experiență fluidă sunt:

1. Monitorizarea constantă a trei stări fundamentale: *mousedown*, *mousemove* și *mouseup*.

Prin utilizarea acestor evenimente, aplicația știe exact când utilizatorul dorește să înceapă o linie, când își mișcă mâna pe suprafața de lucru și când decide să finalizeze trasarea. Fără această supraveghere permanentă, nu am putea face diferența între o simplă mișcare a cursorului și acțiunea propriu-zisă de creație.

2. Pentru precizie maximă, folosim un sistem de coordonate implementat prin funcția *getCanvasPos*.

Aceasta are rolul de a calcula locația exactă a cursorului în raport cu marginile elementului canvas, eliminând erorile care pot apărea din cauza scroll-ului paginii sau a dimensiunii ferestrei browserului. Mai mult, această funcție ține cont de gradul de mărire sau micșorare (zoom), recalibrând poziția punctelor astfel încât linia să apară exact sub vârful cursorului, indiferent de nivelul de detaliu la care lucrăm.

3. Corectarea greșelilor este realizabilă prin intermediul funcției *undoStack*, care funcționează ca o memorie pe termen scurt a aplicației.

De fiecare dată când utilizatorul eliberează butonul mouse-ului, starea actuală a canvas-ului este salvată într-o stivă de date. Această abordare oferă posibilitatea de a reveni rapid la pașii anteriori fără a pierde progresul general, oferind utilizatorului libertatea de a experimenta fără teama de a strica desenul.

4. Funcția de desenare este responsabilă pentru aplicarea în timp real a setărilor de culoare și grosime.

Logica utilizează metoda `ctx.lineTo()` pentru a uni punctele succesive înregistrate în timpul mișcării.

În fragmentul de cod expus, se poate observa cum am integrat și funcția de radieră printr-o structură condițională: dacă unealta activă este setată pe „eraser”, sistemul schimbă automat culoarea de desenare în alb (`#ffffff`), simulând astfel ștergerea prin acoperirea urmelor anterioare.

În caz contrar, se folosește culoarea selectată de utilizator, asigurând o tranziție rapidă între unelte fără a fi nevoie de funcții separate și complicate.

```
if (down) {  
    ctx.lineTo(xPos, yPos);  
    ctx.strokeStyle = currentTool === 'eraser' ? '#ffffff' : color;  
    ctx.lineWidth = width;  
    ctx.stroke();  
}
```

Figură 7. Logica de funcționare

3.2.6 Funcții avansate

Pe lângă instrumentele de bază, am integrat în aplicație o serie de funcționalități avansate care transformă Paint Studio dintr-un simplu creion digital într-un editor grafic versatil. Aceste funcții au necesitat o abordare mai complexă a algoritmilor de calcul și o gestionare atentă a memoriei browserului.

1. Scalare dinamică

Utilizatorii au posibilitatea de a modifica scara de vizualizare a foii de desen prin intermediul butoanelor de tip `+/-`. Această funcție de zoom nu se limitează doar la mărirea vizuală a elementelor, ci presupune o recalculare a tuturor coordonatelor de desenare.

Am implementat o logică prin care, pe măsură ce utilizatorul mărește canvas-ul, rezoluția internă a desenului este păstrată, permițând lucrul la detalii fine fără ca liniile să devină pixelate sau neclare.

2. Adăugarea imaginilor prin lipire

O funcție inovatoare a aplicației este posibilitatea de a importa imagini direct din clipboard. În loc să forțăm utilizatorul să descarce și apoi să încarce un fișier, am programat un event listener pentru comanda *Paste*.

În momentul în care o imagine este lipită pe canvas, aceasta intră într-o stare plutitoare (floating image). Așa cum se poate observa în Figura 8, am dezvoltat un sistem de poziționare care îi permite utilizatorului să mute imaginea oriunde pe suprafața de lucru prin tragere înainte de a o fixa definitiv.

Această manipulare se bazează pe calcularea unui offset între cursor și colțul imaginii, asigurând o mișcare naturală și precisă a obiectului importat.

3. Exportul fișierelor

Ultimul pas în procesul creativ este salvarea rezultatului prin funcția `saveCanvas`. Din punct de vedere tehnic, această metodă convertește întreaga matrice de pixeli de pe canvas într-un format de date numit *base64*, pe care browserul îl poate interpreta ca pe o imagine reală. Datele sunt apoi descărcate automat sub forma unui fișier în format *.png*.

Am ales acest format deoarece păstrează calitatea culorilor și permite transparența, oferind utilizatorului un fișier final gata de a fi folosit în orice alt context digital sau printat, fără pierderi de calitate.

```
if (isDraggingFloat && floatingImage) {  
    var pos = getCanvasPos(touches[0]);  
    floatX = pos.x - dragOffsetX;  
    floatY = pos.y - dragOffsetY;  
    drawFloatingState();  
    return;  
}
```

Figură 8. Mutarea obiectelor pe canvas

4 TESTARE

4.1 VERIFICAREA FUNCȚIONALITĂȚILOR

Pentru a vedea dacă aplicația funcționează corect, am testat manual fiecare buton și instrument în parte. Procesul de testare a fost împărțit în mai multe etape, începând cu verificarea latenței și a fluidității.

Am început prin a desena mai multe linii, cu viteze diferite, pentru a ne asigura că programul ține pasul cu mouse-ul și că nu apar întreruperi sau linii frânte.

De asemenea, am verificat dacă schimbarea culorilor și a grosimii creionului se reflectă imediat ce apăsăm pe butoanele corespunzătoare. Un punct important l-a reprezentat testarea funcției de import prin lipire; am verificat stabilitatea sistemului atunci când mutăm imaginile pe canvas, asigurându-ne că acestea se poziționează exact unde dorim.

Tot la testarea manuală, am verificat cum arată interfața pe diferite rezoluții de ecran și și în diverse browsere pentru a ne asigura că butoanele rămân accesibile și canvas-ul nu iese din cadrul ferestrei. Am urmărit cu atenție stabilitatea aplicației după sesiuni lungi de utilizare, efectuând sute de acțiuni succesive de desenare, ștergere și undo. Acest test de durabilitate a confirmat că programul nu consumă resurse excesive în timp, nu încetinește și nu generează erori vizuale după o utilizare prelungită.

La final, am salvat și exportat mai multe compoziții pe calculator pentru a confirma că procesul de conversie în format .png este unul fidel, rezultând imagini clare care conțin absolut tot ce am lucrat pe ecran.

4.2 TESTAREA AUTOMATĂ

Pe lângă verificările făcute manual, am folosit și un set de teste automate scrise în fișierul *teste.js* pentru a valida logica internă a programului. Aceste teste au permis să verificăm rapid dacă funcțiile matematice și cele de gestionare a datelor funcționează corect, eliminând riscul de a introduce erori noi atunci când modificăm codul.

Sistemul folosește o funcție de validare care verifică dacă valorile din cod corespund cu realitatea, afișând rezultatele direct în consola browserului sub forma unui raport de stare.

Procesul de testare a fost împărțit pe mai multe module logice pentru a acoperi toate scenariile posibile. În prima fază, am verificat starea inițială a aplicației la pornire, asigurându-ne că setările de bază, precum culoarea implicită, grosimea creionului și unealta selectată, sunt încărcate corect la pornire.

Această verificare este critică pentru a preveni comportamentele imprevizibile ale interfeței în momentul în care utilizatorul interacționează pentru prima dată cu canvas-ul.

```
assert("Stare Inițială: Culoarea pornește setată pe negru (#1a1a2e)", color === '■#1a1a2e');  
assert("Stare Inițială: Grosimea pensulei este 5px", width === 5);  
assert("Stare Inițială: Unealta selectată este creionul", currentTool === 'pencil');  
assert("Stare Inițială: Nivelul de Zoom este 1 (100%)", currentZoom === 1);
```

Figură 9. Verificare stări

Principalele elemente verificate automat au fost funcțiile care calculează coordonatele mouse-ului pe suprafața de lucru, asigurându-ne că punctele desenate apar exact sub cursor, indiferent de marginile ferestrei. De asemenea, am testat sistemul de stocare a acțiunilor pentru funcția undo, confirmând că fiecare stare a desenului este salvată și recuperată corect din memorie.

În Figura 10, se observă modul în care am verificat gestionarea memoriei: am simulat un număr mare de acțiuni pentru a confirma că sistemul șterge automat stările foarte vechi și nu depășește limita de 20 de salvări. Această limitare este esențială pentru a preveni pierderile de memorie și pentru a asigura stabilitatea pe termen lung a aplicației, garantând că browserul nu va rămâne fără resurse în timpul unei sesiuni de lucru prelungite.

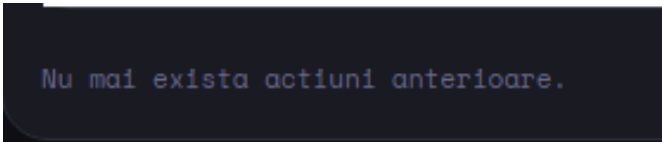
```
for(let i = 0; i < 25; i++) saveCanvasState();  
assert("Undo (Gestionare Memorie): Sistemul șterge automat stările vechi pentru a nu depăși limita de 20 (previne memory leaks)",  
undoStack.length <= 20);
```

Figură 10. Gestionarea memoriei

Așa cum se poate observa în Figura 11, am testat comportamentul funcției de Undo după ce au fost efectuate cele 20 de acțiuni salvate în memorie. În momentul în care utilizatorul încearcă să acceseze o stare care nu mai există, aplicația afișează mesajul „Nu mai există acțiuni anterioare”.

Această abordare este esențială din două puncte de vedere. În primul rând, oferă un feedback vizual clar, prevenind confuzia utilizatorului care ar putea crede că butonul nu funcționează. În al doilea rând, confirmă faptul că logica de curățare a stivei de date funcționează corect, protejând stabilitatea browserului.

Testarea acestui scenariu ne-a asigurat că aplicația PaintStudio rămâne predictibilă și sigură, gestionând elegant situațiile de limită tehnică fără a se bloca sau a genera erori în consola de sistem.



Nu mai exista actiuni anterioare.

Figură 11. Funcția undo după 20 de accesări

Aceste teste automate au fost esențiale pentru a depista greșeli de calcul în cod care nu erau vizibile imediat în aplicație, dar care ar fi putut cauza probleme pe termen lung. La finalizarea execuției scriptului, aplicația generează automat un raport de succes, confirmând integritatea codului.

4.3 REZULTATE

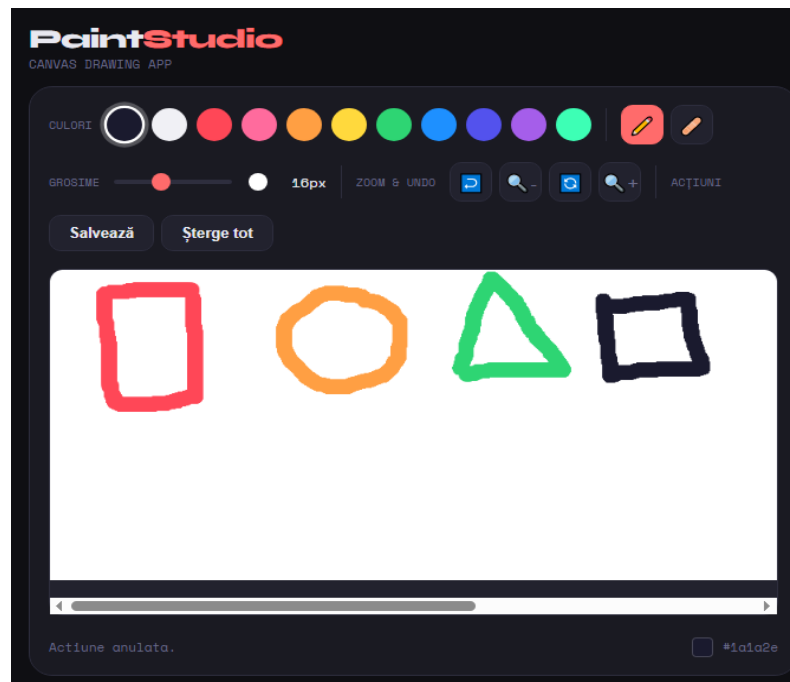
În această etapă prezentăm rezultatele finale ale proiectului, care demonstrează funcționalitatea aplicației, atât din punct de vedere grafic, cât și tehnic.

4.3.1 Testarea grafică

În poza de mai jos putem observa aspectul vizual final, unde am testat claritatea culorilor și fluiditatea liniilor prin trasarea unor forme geometrice simple. Se poate observa că marginile sunt netede, confirmând succesul implementării rotunjirii capetelor de linie.

De asemenea, am testat accesibilitatea butoanelor și a funcțiilor de control. Interfața oferă un feedback vizual imediat: butonul pentru instrumentul activ (creionul) este evidențiat printr-o bordură albă, permițând utilizatorului să știe exact ce unealtă folosește.

În partea de jos a ecranului, bara de stare confirmă ultima acțiune efectuată, oferind un nivel suplimentar de siguranță în timpul utilizării. Toate elementele, de la slider-ul de grosime până la butoanele de zoom, sunt corect aliniate și răspund instantaneu la interacțiune, demonstrând o structură CSS bine definită.



Figură 12. Interfața și testarea grafică

4.3.2 Testarea codului

Poza următoare reprezintă dovada stabilității codului și a calității implementării logice. Am rulat o suită completă de teste unitare care acoperă toate funcțiile critice ale aplicației PaintStudio.

Conform raportului generat direct în consola browserului, toate cele 18 teste efectuate asupra codului au fost reușite, obținând o acoperire de 100%.

Acest raport detaliat confirmă faptul că funcțiile de bază, precum schimbarea culorilor, reglarea grosimii creionului și limitele de mărire/ micșorare, funcționează exact așa cum au fost proiectate, fără erori de calcul. Un succes important este confirmarea logicii pentru funcția de undo și gestionarea memoriei.

Testele arată că sistemul salvează și extrage corect stările din memorie, prevenind în același timp supraîncărcarea sistemului prin ștergerea automată a stărilor vechi. Rezultatul final „18/18 teste trecute” reprezintă garanția că aplicația este stabilă, predictibilă și pregătită pentru o utilizare intensă fără riscul de a se bloca sau de a pierde datele utilizatorului.

[TRECUT] Stare Inițială: Culoarea pornește setată pe negru (#1a1a2e)
[TRECUT] Stare Inițială: Grosimea pensulei este 5px
[TRECUT] Stare Inițială: Unealta selectată este creionul
[TRECUT] Stare Inițială: Nivelul de Zoom este 1 (100%)
[TRECUT] Unelte (Radieră): Variabila și UI-ul trec corect pe Radieră
[TRECUT] Unelte (Creion): Variabila și UI-ul revin corect pe Creion
[TRECUT] Culori (Predefinite): Funcția 'changeColor' actualizează culoarea globală și UI-ul
[TRECUT] Culori (Native Picker): Funcția 'handleNativeColor' procesează corect input-ul
[TRECUT] Culori (Hex Input): Funcția 'applyHexInput' formatează corect codul adăugând '#' automat
[TRECUT] Grosime Pensulă: 'changeSize' setează lățimea contextului 2D la 24px
[TRECUT] Zoom (Incrementare): 'adjustZoom(0.2)' crește nivelul corect
[TRECUT] Zoom (Limita Maximă): Sistemul blochează depășirea limitei de 500% (x5)
[TRECUT] Zoom (Limita Minimă): Sistemul blochează micșorarea sub 20% (x0.2)
[TRECUT] Zoom (Reset): 'resetZoom' aduce corect ecranul la nivelul inițial (1)
[TRECUT] Undo (Salvare): 'saveCanvasState' adaugă corect o stare nouă în memorie
[TRECUT] Undo (Gestionare Memorie): Sistemul șterge automat stările vechi pentru a nu depăși limita de 20 (previne memory leaks)
[TRECUT] Undo (Execuție): 'undoAction' extrage ultima stare din memorie
[TRECUT] Canvas (Ștergere): 'clearCanvas' umple ecranul cu alb și actualizează statusul
REZULTAT FINAL: 18/18 teste trecute. Acoperire: 100%

Figură 13. Raportul final al testelor

În concluzie, testele efectuate confirmă faptul că aplicația este gata pentru utilizare, fiind stabilă și capabilă să gestioneze corect interacțiunile complexe ale utilizatorului.

4.4 MENTENANȚĂ ȘI DEZVOLTARE

Datorită modului modular în care am structurat, aplicația Paint Studio este foarte ușor de întreținut și de extins. Arhitectura bazată pe evenimente ne permite să adăugăm noi unelte sau funcționalități fără a fi nevoie să rescriem întreaga logică a programului. Pe parcursul testării, am identificat câteva direcții clare în care proiectul poate evolua pentru a deveni un editor grafic și mai complex:

1. Sistem de straturi

O dezvoltare viitoare majoră ar fi implementarea unui sistem de straturi, similar cu cel din programele profesionale precum Photoshop. Acest lucru ar permite utilizatorului să deseneze elemente diferite pe „foi” transparente suprapuse, oferind posibilitatea de a modifica un obiect fără a șterge ce se află în spatele lui. Din punct de vedere tehnic, acest lucru s-ar realiza prin suprapunerea mai multor elemente de tip canvas în fișierul HTML.

2. Instrumente de desenare geometrică

Deși în prezent aplicația permite desenarea liberă, o perspectivă de dezvoltare este adăugarea unor unelte pentru forme geometrice perfecte și linii drepte. Logica pentru acestea este deja pregătită în cod, fiind necesară doar implementarea unor funcții matematice care să calculeze coordonatele finale în momentul în care utilizatorul eliberează mouse-ul.

3. Filtre și procesare de imagine

O altă direcție interesantă este adăugarea unor filtre grafice (precum alb-negru sau blur) care să poată fi aplicate instantaneu asupra desenului sau asupra imaginilor importate.

Deoarece folosim Canvas API, avem acces direct la datele fiecărui pixel, ceea ce face implementarea acestor filtre relativ simplă prin manipularea valorilor de culoare (RGB).

4. Colaborare în timp real

Într-o etapă avansată, aplicația ar putea fi conectată la un server pentru a permite mai multor utilizatori să deseneze pe același canvas în același timp.

Această funcționalitate ar transforma Paint Studio dintr-o unealtă individuală într-un spațiu de colaborare creativă, folosind tehnologii precum WebSockets pentru sincronizarea instantanee a liniilor trasate.

Prin aceste perspective, demonstrăm că proiectul actual reprezintă o fundație solidă și scalabilă, capabilă să susțină funcționalități complexe fără a pierde din fluiditatea și simplitatea care îl caracterizează în prezent.

5 CONCLUZII

Realizarea aplicației Paint Studio a reprezentat o oportunitate de a pune în practică și de a consolida cunoștințele de programare web, punând accent pe interacțiunea în timp real prin intermediul tehnologiei HTML5 Canvas.

Rezultatul este o experiență de desenare fluidă și intuitivă, rulată direct în browser, care reușește să integreze cu succes funcționalitățile de bază ale unui editor grafic: desenarea liberă, utilizarea radierei, precum și controlul dinamic al culorilor și al dimensiunilor creionului.

Un punct forte al aplicației este implementarea sistemului de *undo*, care oferă utilizatorului siguranța că poate reveni asupra oricărei greșeli, corectând-o rapid. În completarea acestuia, funcția de mărire permite lucrul la detalii fine fără a pierde din calitatea vizuală a compoziției, asigurând o precizie ridicată în procesul creativ.

Cea mai mare provocare tehnică a fost gestionarea eficientă a stărilor canvas-ului pentru a preveni supraîncărcarea memoriei browserului. Am rezolvat această problemă prin implementarea unei logici de limitare a istoricului de acțiuni, asigurând astfel o performanță constantă chiar și pe sistemele cu resurse hardware modeste.

Această etapă de optimizare a oferit o înțelegere mai profundă a modului în care JavaScript gestionează evenimentele de sistem și alocarea dinamică a resurselor.

Așa cum am detaliat în capitolul dedicat perspectivelor de dezvoltare, Paint Studio reprezintă o bază solidă și scalabilă. Arhitectura modulară permite adăugarea unor funcționalități avansate, precum sistemul de straturi sau unelte pentru forme geometrice, fără a compromite stabilitatea actuală.

Succesul acestui proiect demonstrează viabilitatea ecosistemului web modern; faptul că un instrument complex poate rula fără instalări suplimentare subliniază direcția software-ului actual către portabilitate și performanță ridicată pe orice dispozitiv.

Prin utilizarea exclusivă a tehnologiilor *client-side*, am eliminat dependența de un server extern pentru procesarea grafică, ceea ce reduce la zero latența între mișcarea fizică a mouse-ului și randarea pixelilor. Această arhitectură asigură faptul că aplicația rămâne o soluție sustenabilă, capabilă să funcționeze pe orice configurație care dispune de un motor de randare web actual.

Eficiența algoritmilor utilizați și structura apelurilor către API-ul Canvas demonstrează că este posibilă crearea unui mediu de lucru performant, fără a sacrifica simplitatea sau viteza de încărcare.

În concluzie, Paint Studio este un instrument stabil și eficient, care îndeplinește toate cerințele propuse inițial, fiind rezultatul unui proces riguros de proiectare și testare.

Închei acest proiect cu o imagine a logo-ului aplicației, care sintetizează identitatea vizuală și experiența tehnică acumulată în dezvoltarea acestui instrument de desen digital.



Figură 14. Logo-ul aplicației

6 BIBLIOGRAFIE

- [1] [Mozilla Developer Network](#) (MDN), *HTML5 Canvas API Reference*, [Online]
- [2] [W3Schools](#), *CSS3 Introduction and Selectors*, [Online]
- [3] [Stack Overflow](#) Community, „Questions tagged [canvas] or [javascript]“, [Online]
- [4] [GitHub Inc.](#), „Open source drawing applications repository“, [Online]
- [5] [Chrome for Developers](#), „Chrome DevTools Overview“, [Online]